

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA1522

Register Number	Kanmani L
Name	192511414

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 25

Code of Conduct

I ____ Kanmani L/1922511414____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student: _____

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

1.A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.

Answer:

Problem:

A Hadoop job repeatedly fails because the input splits are uneven and some reducers remain idle.

Likely Design Error:

The main design error is improper data partitioning or data skew. Hadoop divides input data into splits and processes them using multiple mappers. If some splits contain significantly more data than others, certain mappers take much longer to finish. As a result, the workload is not distributed evenly. Similarly, if the partitioning of intermediate data is unbalanced, some reducers receive a large amount of data while others remain idle.

Corrective Actions:

- Choose an appropriate input split size based on the data volume and cluster capacity.
- Use a suitable partitioner to distribute intermediate key-value pairs evenly among reducers.
- Identify and handle skewed keys that cause excessive data to be sent to a particular reducer.
- Adjust the number of reducers according to the workload.
- Use techniques such as combiners to reduce the amount of intermediate data transferred.
- Monitor mapper and reducer execution times to identify workload imbalance.

The problem can be solved by improving data partitioning, handling data skew, and properly configuring the number of mappers and reducers. This ensures better parallelism and prevents reducers from remaining idle.

2.An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.

Answer:

Problem:

An HDFS deployment experiences frequent data unavailability whenever a single DataNode fails.

Probable Design Error:

The most likely problem is an insufficient replication factor or poor replica placement. HDFS is designed to store multiple copies of each data block. If only one copy exists, failure of its DataNode makes the data unavailable. Even with replication, poor placement of replicas can cause problems if multiple replicas are located on the same physical rack or failure domain.

Another possible issue is the use of a single NameNode without High Availability (HA). Since the NameNode maintains HDFS metadata, its failure can make the entire file system inaccessible even when the actual data blocks still exist.

Corrective Actions:

- Set an appropriate replication factor, commonly 3 for important data.
- Distribute replicas across different DataNodes and racks.
- Enable NameNode High Availability using Active and Standby NameNodes.
- Configure automatic re-replication when a DataNode fails.
- Regularly monitor block replication and DataNode health.
- Use rack-aware placement to avoid losing multiple replicas due to a rack failure.

Proper block replication, rack-aware replica placement, and NameNode HA eliminate single points of failure and improve HDFS availability.

3.A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.

Answer:

Problem:

A data ingestion workflow using ETL and Apache NiFi produces duplicate records in the analytics database.

Likely

Causes:

The duplicates may occur because the same input file or message is processed more than once. This can happen when a processor retries an operation after a temporary failure or when the workflow does not properly track already processed data.

Another important cause is the absence of idempotency and duplicate detection. If the destination database simply inserts every received record without checking whether it already exists, repeated processing will create duplicate records.

Recommended Fixes:

- Use NiFi's DetectDuplicate processor when suitable for the workflow.
- Generate or identify each record using a unique identifier.
- Configure proper retry and failure-handling mechanisms in NiFi.
- Use database primary keys or UNIQUE constraints to prevent duplicate insertion.
- Design ETL operations to be idempotent, meaning processing the same record multiple times produces the same final result.
- Maintain tracking information for successfully processed files or messages.
- Validate the data before loading it into the analytics store.

Example:

If a transaction has `Transaction_ID = T1001`, the database should reject another record with the same transaction ID rather than inserting it again.

Duplicate records can be prevented by combining NiFi duplicate detection, unique identifiers, proper retry handling, and database constraints.

4.A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.

Answer:

Problem:

A YARN cluster experiences resource starvation when several users submit jobs at the same time.

Likely Design Error:

The problem is usually caused by poor resource scheduling and allocation. If one user or application consumes most of the available CPU and memory, other applications may not receive sufficient resources. This can happen when queues, user limits, and application priorities are not properly configured.

Better Resource Management Approach:

- Use Capacity Scheduler to allocate guaranteed resources to different queues.
- Alternatively, use Fair Scheduler where resources are distributed more fairly among applications/users.
- Create separate queues for different teams, departments, or workloads.
- Set maximum and minimum resource limits for each queue.
- Configure user/application limits to prevent a single user from consuming the entire cluster.
- Set suitable application priorities for critical jobs.
- Monitor CPU, memory, containers, and queue utilization.
- Dynamically adjust resource allocation based on workload requirements.

Example:

Suppose a cluster has 100 GB memory. Instead of allowing one user's job to consume all 100 GB, separate queues could be configured so that different departments receive controlled portions of the available resources.

Using queue-based scheduling, resource limits, priorities, and continuous monitoring prevents resource starvation and provides fair resource sharing among users.

5.A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Answer:

Problem:

A distributed-storage system restores outdated data after a failure.

Probable Design Flaw:

The backup system is probably taking backups too infrequently, or the recovery process is using an old backup or stale replica. Another design mistake is treating replication as a complete backup solution. Replication provides availability, but if data is accidentally deleted or corrupted, that change may also be replicated to other copies.

Corrective Actions:

- Perform backups at regular and sufficiently frequent intervals.
- Use incremental backups to capture changes between full backups.
- Maintain versioned snapshots so that earlier versions can be restored.
- Monitor replication lag between primary and replica systems.
- Maintain multiple backup copies in separate storage locations.
- Follow the 3-2-1 backup principle where appropriate: multiple copies, different storage media, and at least one off-site copy.
- Regularly perform restore tests to verify that backups are usable and current.
- Clearly define Recovery Point Objective (RPO) and Recovery Time Objective (RTO).

Example:

If the system is backed up only once a week and a failure occurs six days after the last backup, the recovered system may lose up to six days of recent changes.

Frequent versioned backups, synchronized replication, off-site copies, and regular recovery testing ensure that the latest valid data can be restored after a failure.